OpenTPMS — Firmware Bring-Up & PPK2 Guide

nRF52-DK toolchain to working sensor drivers · Power Profiler Kit II setup · macOS

Guide version 2026-08-15 · Companion to `docs/implementation-plan.md` (Tasks 1–9) and `docs/assembly-guide.html`

> **Honest starting point:** every file in `firmware/src/` is a TODO stub — there is no working firmware yet. That's fine: the stubs define the module structure, the implementation plan defines the tasks, and this guide gets you from empty bench to writing them. The DK lets you do 80% of firmware work before a single sensor board is assembled.

Part A — Firmware bring-up on the nRF52-DK

A1 · Install the toolchain (one afternoon)

Nordic's tooling changed since our docs were written: **`nrfjprog` is deprecated** — its replacement is `nrfutil device`. The stack you want on a Mac in 2026:

Tool	What for	Get it
SEGGER Embedded Studio (SES)	IDE + compiler + debugger. Free for Nordic chips (license auto-activates for nRF targets). This is the path of least resistance for nRF5 SDK work	segger.com downloads → Embedded Studio for ARM, macOS package
J-Link Software Pack	Debug probe drivers + **J-Link RTT Viewer** (your printf window)	segger.com → J-Link software (installs alongside SES)
nRF Util	Command-line flash/recover/erase (replaces nrfjprog)	`brew install --cask nrfutil`, then `nrfutil install device`
nRF Connect for Desktop	Launcher for the Power Profiler app (Part B) and Programmer GUI	nordicsemi.com → nRF Connect for Desktop, macOS
nRF5 SDK 17.1.0	The SDK our firmware targets (zip, not a package manager)	nordicsemi.com → nRF5 SDK downloads → 17.1.0

> **Unzip the SDK OUTSIDE the repo** — e.g. `~/nordic/nRF5_SDK_17.1.0_ddde560`. It's ~500MB of Nordic code that never goes in git. Our Makefile/SES project will reference it by path.

> **Why nRF5 SDK and not the newer nRF Connect SDK (Zephyr)?** Concurrent BLE+ANT on nRF52832 runs on the S332 SoftDevice, which belongs to the classic nRF5 SDK world. The nRF5 SDK is in maintenance mode but fully functional and stays the right choice for this chip + ANT combination. Don't mix tutorials from the two ecosystems — if a tutorial says `west build` or "Zephyr," it's the wrong SDK.

> **SES 8.x + SDK 17.1 compatibility (verified on SES 8.30a, 2026-08-22):** the SDK's example projects predate SES 8 and fail to link out of the box with `.text section is larger than specified size` / same for `.rodata`. Two one-time fixes:
>
> 1. Add `arm_linker_variant="GNU"` to the `<configuration Name="Common">` block of the `.emProject` (SES 8 defaults to the SEGGER linker; the SDK expects GNU ld).
> 2. In the project's `flash_placement.xml`, delete the bogus `size="0x4"` attribute from the `.text` and `.rodata` ProgramSection lines — old linkers ignored it, SES 8's enforces it as a 4-byte cap.
> 3. `SEGGER_RTT_Syscalls_SES.c` fails with `__vfprintf.h: No such file or directory` — the SDK bundles an old SEGGER file that includes a header removed from modern SES runtimes. Fix: wrap the file's contents in a `__has_include("__vfprintf.h")` guard so it compiles to nothing on SES 8 (the file only retargeted printf to RTT; NRF_LOG doesn't need it). One shared copy at `external/segger_rtt/` covers all examples.
>
> And two more that hit every fresh setup regardless of SES version, the moment you build an **s332-variant** project:
>
> 4. **`ble.h: No such file or directory`** — the SDK ships the `components/softdevice/s332/` scaffold but legally cannot bundle Garmin's headers. Marry them in yourself from the thisisant download: copy the API `include/*.h` files into `components/softdevice/s332/headers/` (flat, mirroring the s132 layout — the `nrf52/nrf_mbr.h` subfolder comes along) and drop the SoftDevice hex in `s332/hex/`.
> 5. **`#error "You must obtain a valid license key to use ANT..."`** — deliberate license gate in `s332/headers/nrf_sdm.h`. For non-commercial prototypes, uncomment the `ANT_LICENSE_KEY` evaluation-key line directly above the #error. Commercial sales require the paid key ($0.08/unit royalty, $800/6-mo minimum — see PROD notes).
>
> Both can be applied across the whole SDK in one shell pass (every `.emProject` and every `flash_placement*.xml`). Also: brand-new chips ship with readback protection — the first flash needs `nrfutil device recover` once.

A2 · Day 1: prove the hardware loop (blink)

1. Plug the DK into USB. A drive named `JLINK` appears — that's the on-board debugger. Power switch ON (it's easy to miss, bottom left).
2. In SES: *File → Open Solution →* `SDK/examples/peripheral/blinky/pca10040/blank/ses/blinky_pca10040.emProject`. (`pca10040` = the nRF52-DK board; `blank` = no SoftDevice.)
3. Build (⌘B), then *Target → Download*. LEDs 1–4 chase on the board.
4. Sanity-check the CLI path too: `nrfutil device list` should show the DK's J-Link serial.

That's the whole development loop: edit → build → flash → observe. Everything else is content.

A3 · Day 1–2: RTT logging (your debug lifeline)

RTT streams printf-style logs through the debugger — no UART wiring, and unlike a UART it works on the sealed sensor later via the SWD pads.

1. **Heads-up: almost no SDK example ships with RTT logging on.** Most have `NRF_LOG_ENABLED 0` or log to UART. Best first target is **twi_scanner** (you need it anyway as the board acceptance test): in `twi_scanner/pca10040/blank/config/sdk_config.h` set `NRF_LOG_BACKEND_RTT_ENABLED 1` and `NRF_LOG_BACKEND_UART_ENABLED 0` (NRF_LOG_ENABLED is already 1).
2. Build + Download, then open **J-Link RTT Viewer**: Connection USB, device NRF52832_XXAA, SWD 4000kHz, RTT control block auto. If it won't connect, SES is holding the J-Link — *Target → Disconnect* first.
3. Firmware output is in the **Terminal 0 tab**, not the log tab. If it's empty, press the board's RESET button — the app logged at boot before the viewer attached, and each RESET replays it.
4. Expected from twi_scanner on a bare DK: `TWI scanner started.` / `No device was found.` — the empty-bus result IS the pass. The same binary against assembled board #1 must find **0x76 and 0x19**.

A4 · The S332 SoftDevice (BLE + ANT) — account required

1. The S332 is distributed by Garmin/ANT, not bundled with the SDK. Create a free account at **thisisant.com** (Adopter level), then download the **S332 SoftDevice for nRF52832** matching SDK 17 (the SDK's ANT examples state the compatible S332 version in their release notes — take whatever the download page pairs with SDK 17.x).
2. While logged in, also grab the **ANT+ network key** (free for adopters) — you'll need it the day `ant_tpms.c` broadcasts on the ANT+ network.
3. License note: S332 ships with an evaluation key that's fine for prototypes/non-commercial. Selling sensors later requires the commercial ANT license — already on the project radar (~$100/yr, design-spec §5).
4. Flash the SoftDevice hex once: `nrfutil device program --firmware s332_nrf52_X.X.X_softdevice.hex` (or SES does it automatically for SoftDevice-based projects).
5. Prove BLE: build an S332-variant example (or `ble_app_template` adapted to S332), flash, and find your DK advertising in **nRF Connect** on your phone. Prove ANT: `examples/ant/ant_broadcast`.

A5 · I2C groundwork before sensors exist

Your MS5837 and LIS2DH12 are bare SMD parts — nothing breadboardable. You can still get the entire I2C stack ready:

1. Build and flash `examples/peripheral/twi_scanner`. It scans the bus and logs every device found. Today it finds nothing — that's expected. This exact binary becomes your **board #1 acceptance test**: run it against the assembled sensor via SWD and it must report `0x76` and `0x19`.
2. If you want a live test sooner: any I2C breakout from the parts bin (the ADS1115 boards from the air-station project are I2C! address 0x48) wired to the scanner's SDA/SCL pins + 3V + GND proves your TWIM config end-to-end.
3. Read the TWIM (TWI Master with EasyDMA) driver docs in the SDK — that's the API our drivers use (design-spec §3 power notes explain why: CPU sleeps during transfers).

A6 · Writing the actual firmware — recommended order

The repo stubs map to implementation-plan tasks. Write them in this order — it front-loads everything testable *without* the sensor board:

#	Module	Hardware needed	Notes
1	`ms5837.c` conversion math	None (host-compiled)	The 24-bit raw → mbar/°C compensation math from the TE datasheet is pure arithmetic. Write it + `tests/test_ms5837.c` against the datasheet's worked example values, run on your Mac with plain `cc`. Classic first-embedded-project win
2	`calibration.c` + `alert.c`	None	Polynomial correction and ideal-gas-law flat detection — pure logic, unit-test both (`tests/` stubs exist)
3	`ms5837.c` I2C layer + `motion.c`	DK only (compiles/logs), board #1 to verify live	TWIM transactions; LIS2DH12 register setup for the motion-detect interrupt (ST's app notes have the exact register recipe)
4	`storage.c` (FDS)	DK	Nordic Flash Data Storage for location/threshold/calibration — fully testable on the DK
5	`ble_tpms.c` GATT service	DK + phone	Custom service per design-spec §3.5 — test with nRF Connect app, then the web-config page
6	`ant_tpms.c`	DK + your Garmin	Broadcast per the TubeWitch-agreed format: **byte 2 = 0x04, bytes 3–5 = FF FF FF** (see tubewitch-reply3). Verify on the Edge
7	`main.c` state machine + WDT + SAADC + DFU	DK, then board #1	Sleep/wake per design-spec §3; power-tune with the PPK2 (Part B)

> **Rule of thumb for the whole phase:** if a bug can be found on the Mac (math) or the DK (drivers, radio), find it there. Board #1's job is confirming I2C wiring and power numbers — not debugging logic.

Part B — PPK2 setup & first measurements

B1 · Setup (20 minutes)

1. Open **nRF Connect for Desktop** → install the **Power Profiler** app.
2. Connect the PPK2 with a **data** USB cable (a charge-only cable is the #1 "it doesn't work" cause).
3. The app will prompt to update the PPK2's firmware on first connect — accept.
4. Two modes, one decision:
    - **Source meter** — PPK2 *supplies* 0.8–5.0V and measures the current it delivers. **This is our mode**: it plays the role of the CR1225 for bench work.
    - **Ampere meter** — PPK2 sits in series with an external supply. You won't need it until measuring a battery-powered sealed unit.

B2 · Practice 1: known resistor (5 minutes, no surprises)

1. Source meter mode, 3.0V, output enabled, a 10kΩ resistor from VOUT to GND (clip leads or breadboard).
2. Expect a flat line at **~300µA** (Ohm's law: 3.0V/10k). If you see that, you understand the tool.
3. Play with the UI on this boring signal: log vs linear Y axis, zoom, the averaging window selection, and the live average readout. These are the controls you'll use constantly.

B3 · Practice 2: a real MCU profile (optional but worth it)

The DK has a current-measurement header (P22) that lets the PPK2 measure just the nRF52832 — the DK user guide's "current measurement" section shows the jumper arrangement. Flash a copy of blinky, then a copy where the CPU sleeps between blinks (`__WFE()`), and look at the difference. Seeing an MCU's personality in the current trace — CPU spikes, sleep floors, radio bursts later — is the skill that makes the power budget real.

B4 · What you'll expect from the real board (reference)

State	Expected (LDO)	Expected (DC-DC, if L1 works)
Blank chip, first power-up	µA to low-mA, stable. Tens of mA = short, power off	
Deep sleep (System OFF + accel watch)	~3µA	~3µA
Idle between TX (RTC running)	~5µA	~3µA
BLE TX burst (~3ms every 3s)	~12mA peak	~8mA peak
ANT+ TX burst (~2ms every 3s)	~8mA peak	~5.3mA peak
Average while riding (target)	<20µA → 2–3 years on 48mAh	

> **The L1 verdict, measured:** once board #1 runs TX firmware, capture a trace with `DCDCEN` off, then on. If the TX peaks drop ~30%, the overhang-soldered 0603 inductor works — record it in the BOM. If current goes weird or the radio glitches, remove L1 and ship Rev 1 as LDO.

B5 · PPK2 habits worth forming now

- Save a screenshot/export of every meaningful trace into `docs/images/power/` — before/after comparisons are the whole game in power tuning, and they make great README material.
- Measure at 3.0V (fresh cell) and once at 2.4V (dying cell) before calling power "done" — brownout behavior at TX peaks is a cold-weather failure mode we specifically designed the 220µF buffer against.
- Sleep-current debugging order when the floor is too high: flux residue on the board → a peripheral left enabled (TWIM, UART) → log backend still on → GPIO floating. The PPK2 can't tell you which, but the magnitude narrows it.

Milestones for this phase

#	Milestone	Proof
1	Toolchain alive	Blinky flashed from SES; `nrfutil device list` sees the DK
2	Logging alive	RTT Viewer streaming NRF_LOG output
3	S332 alive	DK advertising over BLE (phone sees it) + ANT broadcast example runs
4	Math proven	MS5837 compensation + calibration + alert unit tests pass on the Mac
5	I2C ready	twi_scanner runs; (optional) finds an ADS1115 at 0x48
6	PPK2 fluent	Resistor test done; a saved trace exists
7	Ready for board #1	Everything above ✓ — assembly guide takes over from here

 · Task details in `docs/implementation-plan.md` · ANT+ packet format agreement in `tubewitch-reply3.txt` (byte 2 = 0x04).